

RAG System Architecture Decision Record

ADR-001 | Status: Accepted | Author: Platform Engineering | Date: 2026-06-21

1. Context

The organisation needs a production-grade Retrieval-Augmented Generation (RAG) system to answer employee questions from a corpus of HR, legal, and technical documents. Two candidate architectures were evaluated: a PostgreSQL-only approach using the pgvector extension, and a hybrid approach combining PostgreSQL with a dedicated vector database (Qdrant).

2. Architecture Comparison

2.1 Vector Index Performance

Metric	pgvector (HNSW)	Qdrant (HNSW + BQ)	Winner
QPS at 1M vectors	~800 QPS	~6,500 QPS	Qdrant
Memory (1M × 768-dim)	~3.0 GB	~95 MB (binary quant)	Qdrant
ANN Recall@10	96.2%	95.8% (with rescore)	pgvector
Sparse vector support	No	Yes (SPLADE)	Qdrant
Payload filtering	Post-filter only	Pre-filter (no recall drop)	Qdrant
Operational complexity	Low (1 service)	Medium (2 services)	pgvector
Horizontal scaling	Via PG replicas	Native distributed mode	Qdrant

Table 1: Vector index performance comparison at 1M chunks, 768 dimensions.

2.2 Hybrid Search Fusion

Hybrid search combining dense (semantic) and sparse (keyword) retrieval with Reciprocal Rank Fusion (RRF) improves recall by 12–18% versus dense-only search on the BEIR benchmark. pgvector does not support sparse vectors natively, requiring a separate BM25 implementation and manual RRF fusion in SQL. Qdrant implements both dense and sparse vectors natively with RRF fusion in a single query API call.

3. Decision

We select the **Qdrant Hybrid Architecture (v2)** for production deployment. PostgreSQL remains the source of truth for all relational data (document text, metadata, audit trail). Qdrant serves as the vector index only. The chunk_id UUID is the join key between systems.

3.1 Data Flow

The following pseudocode illustrates the ingestion and query data flows:

```
# INGESTION FLOW
parse(file) -> ParsedDocument
chunk(doc) -> List[Chunk] # SmartChunker:
table-safe embed(chunk) -> (dense_768, sparse) # nomic-embed-text + SPLADE
pg.insert(chunk)
# PostgreSQL: source of truth
qdrant.upsert(chunk_id, vectors) # Qdrant: vector index
# QUERY
```

```
FLOW embed(query) -> (dense_768, sparse) results = qdrant.hybrid_search( # RRF fusion native
prefetch_dense=50, prefetch_sparse=50, top_k=10 ) chunks = pg.hydrate(results.ids) # batch
SELECT by UUID answer = llm.generate(query, chunks)
```

4. Consequences

Concern	Impact	Mitigation
Two systems to maintain	Higher ops overhead	Reconciler runs nightly; PG is always recoverable source
Dual-write consistency	PG + Qdrant can diverge	Write PG first; qdrant_synced flag; reconciler fixes drift
Redis semantic cache adds Ops	On-call service	Redis already present for Celery; reuses existing infra
Qdrant learning curve	Onboarding overhead	See docs/ONBOARDING.md and Qdrant docs
Network latency PG→Qdrant	10ms per query	gRPC port 6334; batched hydration; cache hits skip both

5. Alternatives Rejected

pgvector-only (v1): Simpler operationally but 8× slower ANN throughput at scale, no native sparse vectors, no pre-filtering, and 32× higher memory usage. Adequate for corpora under 100K chunks. Documented in docs/PRODUCTION_RAG_SYSTEM.md.

Pinecone (managed): Excellent performance but data leaves premises. Not acceptable for confidential HR and legal documents under current data governance policy (see docs/DATA_GOVERNANCE.md).

ChromaDB: Suitable for prototyping. Lacks binary quantization, distributed mode, SPLADE support, and production observability features needed at target scale.